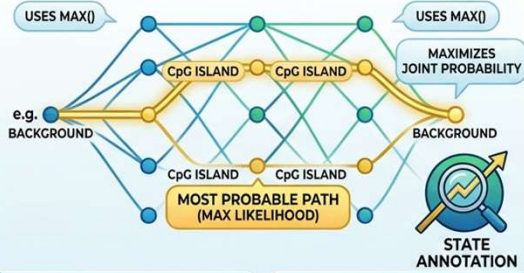


COMPARING HMM ALGORITHMS

DNA SEQUENCE (OBSERVED EMISSIONS)

**VITERBI ALGORITHM: PATH FINDING (DECODING)**

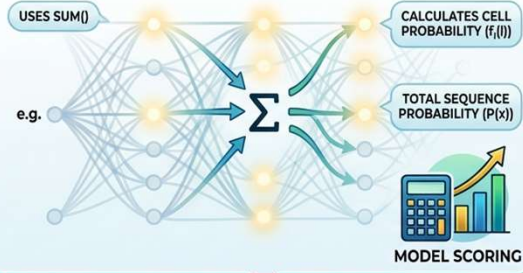
GOAL: FIND THE SINGLE MOST PROBABLE STATE PATH



BG-BG-CpG-CpG-BG
ANNOTATED PATH

FORWARD ALGORITHM: SCORE CALCULATION (EVALUATION)

GOAL: SUM PROBABILITIES OF ALL POSSIBLE PATHS



0.00394
TOTAL SCORE ($P(x)$)

Lecture 10**Coding and Non-Coding Genes
HIDDEN MARKOV MODELS II
Comparing HMM algorithms**

Ciele

Kľúčové slová

Literatúra

1. Algoritmické nastavenia pre HMM

Algoritmické nastavenia pre HMM

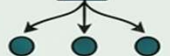
HMM používame na tri typy operácií:

- skórovanie,
- dekodovanie a
- učenie.

V tejto prednáške budeme hovoriť o **skórovaní a dekodovaní**.

Každá z týchto operácií sa môže vykonávať pre jednu cestu alebo cez všetky možné cesty. Operácie pre jednu cestu sa zameriavajú na objavenie cesty s maximálnou pravdepodobnosťou, zatiaľ čo pri všetkých cestách nás zaujíma sekvencia pozorovaní alebo emisií bez ohľadu na konkrétne zodpovedajúce cesty.

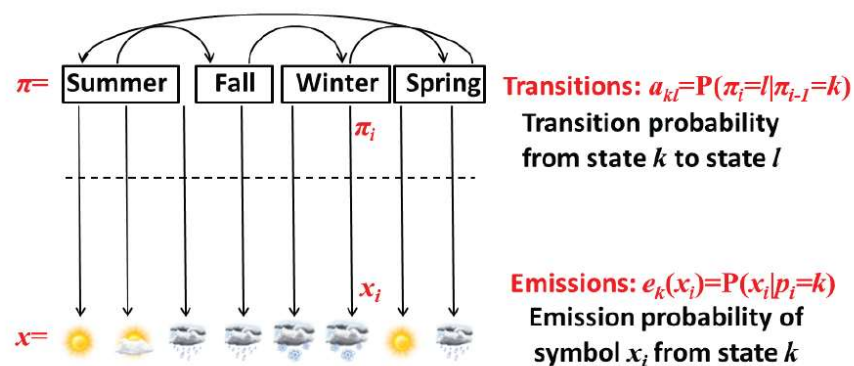
1. Algoritmické nastavenia pre HMM

Hidden Markov Models: Operations Matrix 	
	One Path All Paths
Scoring	1. Scoring $\mathbf{x}$, one path  $P(\mathbf{x}, \pi)$  Prob of a path, emissions 2. Scoring $\mathbf{x}$, all paths  $P(\mathbf{x}) = \sum_{\pi} P(\mathbf{x}, \pi)$  Prob of emissions, over all paths
Decoding	3. Viterbi decoding  $\pi^* = \operatorname{argmax}_{\pi} P(\mathbf{x}, \pi)$ Most likely path 4. Posterior decoding  $\pi^{\wedge} = \{\pi_i \mid \pi_i = \operatorname{argmax}_{\pi} \sum_{\pi} P(\pi_i = k \mid \mathbf{x})\}$ Path containing the most likely state at any time point.
Learning	5. Supervised learning, given π  $\Lambda^* = \operatorname{argmax}_{\Lambda} P(\mathbf{x}, \pi \mid \Lambda)$ 6. Unsupervised learning.  $\Lambda^* = \operatorname{argmax}_{\Lambda} \max_{\pi} P(\mathbf{x}, \pi \mid \Lambda)$ Viterbi training, best path 6. Unsupervised learning  $\Lambda^* = \operatorname{argmax}_{\Lambda} \sum_{\pi} P(\mathbf{x}, \pi \mid \Lambda)$ Baum-Welch training, over all paths

1. Algoritmické nastavenia pre HMM

Formalizácia Markovovho reťazca a HMM

Aby sme plne porozumeli Skrytým Markovovým modelom - HMM, musíme definovať kľúčové parametre, ako je znázornené na obrázku.



Vektor x predstavuje sekvenciu pozorovaní (emisíí).

Vektor π predstavuje skrytú cestu, čo je sekvencia skrytých stavov.

Každý záznam a_{kl} matice prechodov A označuje pravdepodobnosť prechodu zo stavu k do stavu l .

Každý záznam $e_k(x_i)$ emisného vektora označuje pravdepodobnosť pozorovania x_i zo stavu k .

Napokon, s týmito parametrami a Bayesovým pravidlom môžeme použiť známe

$$p(x_i | \pi_i = k) \text{ na odhad neznámej pravdepodobnosti } p(\pi_i = k | x_i).$$

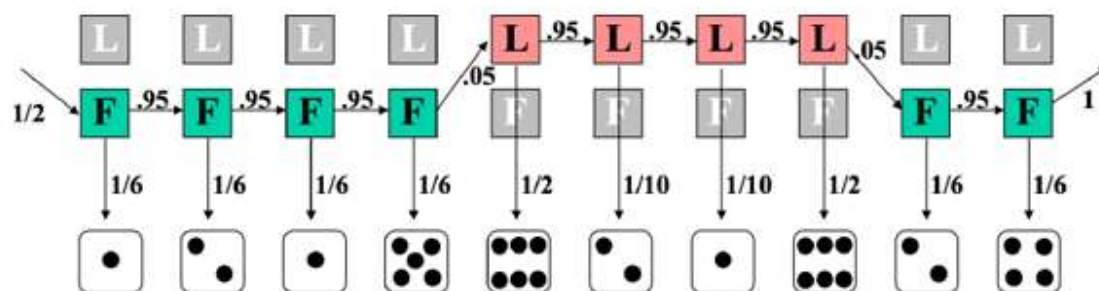
1. Algoritmické nastavenia pre HMM

Skórovanie

Skórovanie cez jednu cestu

Problémy „Nepoctivé kasíno“ a predpoveď oblastí bohatých na GC z minulej prednášky sú obidva príklady **hľadania pravdepodobnosti jednej cesty**. Pre tento prípad definujeme problém skórovania nasledovne:

- Vstup:** Sekvencia pozorovaní $x = x_1 x_2 \dots x_n$ generovaná HMM $M(Q; A; p; V; E)$ a cesta stavov $\pi = \pi_1 \pi_2 \dots \pi_n$.



- Výstup:** Združená pravdepodobnosť $P(x, \pi)$ pozorovania x , ak je sekvencia skrytých stavov π . Výpočet pre jednu cestu v podstate používa nasledujúci vzorec:

$$P(x, \pi) = P(x | \pi) P(\pi)$$

1. Algoritmické nastavenia pre HMM

Skórovanie cez všetky cesty

Verziu problému skórovania pre všetky cesty definujeme nasledovne:

- **Vstup:** Sekvencia pozorovaní $x = x_1 x_2 \dots x_n$ generovaná HMM $M(Q; A; p; V; E)$.
- **Výstup:** Združená pravdepodobnosť $P(x, \pi)$ pozorovania x cez všetky možné sekvencie skrytých stavov π .

Pravdepodobnosť danej sekvencie pozorovaní cez všetky cesty skrytých stavov π je daná nasledujúcim vzorcom:

$$P(x) = \sum_{\pi} P(x, \pi)$$

Toto skóre používame vtedy, keď nás

zaujíma vierohodnosť konkrétnej sekvencie pre daný HMM.

Naivný výpočet tohto súčtu však vyžaduje **výpočet exponenciálneho počtu možných ciest**.

Neskôr v prednáške uvidíme, ako vypočítať túto veličinu v polynomiálnom čase.

1. Algoritmické nastavenia pre HMM

Dekódovanie

Problémom dekodovania je určiť, vzhľadom na nejakú pozorovanú sekvenciu, ktorá cesta nám dáva maximálnu vierohodnosť pozorovania tejto sekvencie?

Formálne definujeme problém nasledovne:

- **Dekódovanie cez jednu cestu:**
 - **Vstup:** Sekvencia pozorovaní $x = x_1 x_2 \dots x_N$ generovaná HMM $M(Q; A; p; V; E)$.
 - **Výstup:** Najpravdepodobnejšia cesta stavov, $\pi^* = \pi_1^* \pi_2^* \dots \pi_N^*$.
- **Dekódovanie cez všetky cesty:**
 - **Vstup:** Sekvencia pozorovaní $x = x_1 x_2 \dots x_N$ generovaná HMM $M(Q; A; p; V; E)$.
 - **Výstup:** Cesta stavov, $\pi^* = \pi_1^* \pi_2^* \dots \pi_N^*$, ktorá obsahuje najpravdepodobnejší stav v každom časovom bode.

V tejto prednáške sa pozrieme iba na problém dekodovania cez jednu cestu. Problém dekodovania cez všetky cesty bude diskutovaný v ďalšej prednáške.

Ako bolo spomenuté skôr, **prístup hrubou silou k problému jednej cesty zahŕňa výpočet združených pravdepodobností danej sekvencie emisií a všetkých možných ciest. Exponenciálny** počet možností robí tento prístup nepraktickým. Na vyriešenie tohto problému možno použiť dynamické programovanie.

1. Algoritmické nastavenia pre HMM

Chceli by sme nájsť najpravdepodobnejšiu sekvenciu stavov na základe pozorovania.

Ako vstupy sú nám dané parametre modelu

- $e_t(s)$, emisné pravdepodobnosti pre každý stav, a
- a_{ij} , pravdepodobnosti prechodu.
- Daná je aj sekvencia emisií x .
- **Cieľom je nájsť sekvenciu skrytých stavov π^* , ktorá maximalizuje združenú pravdepodobnosť pre danú sekvenciu emisií.** Teda:

$$\pi^* = \arg \max_{\pi} P(x, \pi)$$

Vzhľadom na emitovanú sekvenciu x môžeme vyhodnotiť akúkoľvek cestu cez skryté stavy. **My však hľadáme najlepšiu cestu.**

Začíname hľadaním optimálnej podštruktúry tohto problému.

Môžeme povedať, že najlepšia cesta cez daný stav musí obsahovať nasledujúce:

- Najlepšiu cestu k predchádzajúcemu stavu
- Najlepší prechod z predchádzajúceho stavu do aktuálneho stavu
- Najlepšiu cestu ku koncovému stavu

Preto možno najlepšiu cestu získať na základe najlepšej cesty predchádzajúcich stavov, t. j. **najlepšiu cestu môžeme nájsť rekurzívne.**

Viterbiho algoritmus je prístup dynamického programovania, ktorý sa bežne používa na vyriešenie tohto problému.

1. Algoritmické nastavenia pre HMM

Viterbiho algoritmus

Predpokladajme, že $v_k(i)$ je známa pravdepodobnosť, že najpravdepodobnejšia cesta končí v pozícii (alebo časovej inštancii) i v stave k , pre každé k . Potom môžeme vypočítať zodpovedajúce pravdepodobnosti v čase $i+1$ pomocou nasledujúcej rekurzii:

$$v_i(i+1) = e_i(x_{i+1}) \max_k (a_{ki} v_k(i))$$

Najpravdepodobnejšia cesta π^* , alebo maximum $P(x, \pi)$, sa dá nájsť rekurzívne a nová najpravdepodobnejšia cesta pre každý stav závisí od:

- Maximálneho skóre predchádzajúcich stavov
- Pravdepodobnosti prechodu
- Pravdepodobnosti emisie

Inými slovami, nové maximálne skóre pre konkrétny stav v čase i je to, ktoré maximalizuje nasledujúce: penalizáciu prechodu z každého predchádzajúceho stavu vynásobenú ich maximálnymi predchádzajúcimi skóre vynásobenú pravdepodobnosťou emisie v aktuálnom čase.

1. Algoritmické nastavenia pre HMM

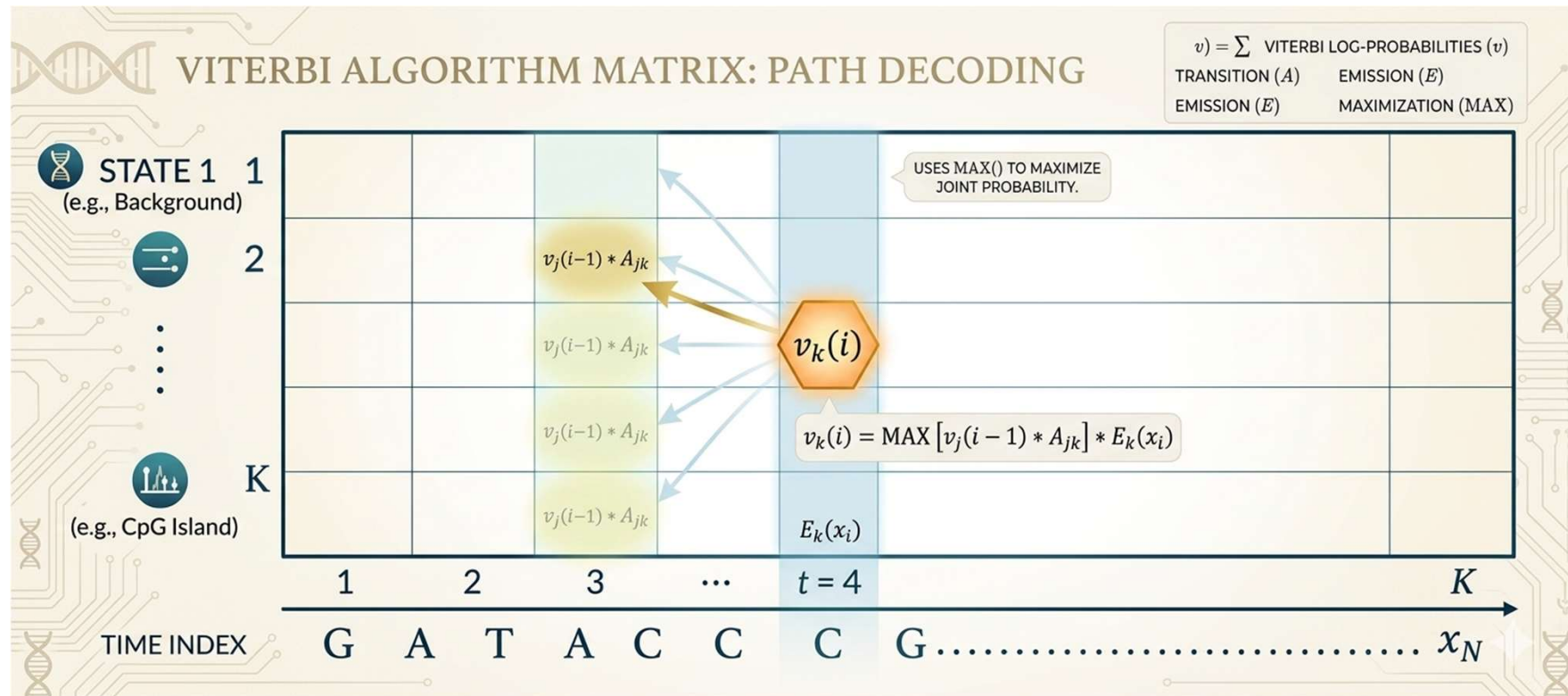
Všetky sekvencie **musia začať v stave 0** (počiatočný stav). **Sledovaním ukazovateľov možno následne nájsť skutočnú sekvenciu stavov pomocou spätného sledovania (backtracking)**. Riešenie tohto problému dynamického programovania je veľmi podobné riešeniu zarovnávacích algoritmov, ktoré boli prezentované v predchádzajúcich prednáškach.

Kroky Viterbiho algoritmu sú zhrnuté nižšie:

1. **Inicializácia** ($i = 0$): $v_0(0) = 1, v_k(0) = 0$ pre $k > 0$.
2. **Rekurzia** ($i = 1 : : N$): $v_k(i) = e_k(x_i) \max_j (a_{jk} v_j(i-1))$; $ptr_i(i) = \arg \max_j (a_{jk} v_j(i-1))$.
3. **Ukončenie**: $P(x, \pi^*) = \max_k v_k(N)$; $\pi_N^* = \arg \max_k v_k(N)$.
4. **Spätné sledovanie** ($i = N : : 1$): $\pi_{i-1}^* = ptr_i(\pi_i^*)$.

Ako je znázornené na obrázku, vyplňame maticu zľava doprava a spätne sledujeme. Každá pozícia v matici má K stavov, ktoré treba zvážiť, a v matici je K^N buniek, takže požadovaný výpočtový čas je $O(K^2N)$ a požadovaný priestor je $O(KN)$ (na zapamätanie ukazovateľov). **Všimnite si, že čas behu bol znížený z exponenciálneho na polynomiálny.** V praxi používame na výpočet logaritmy pravdepodobností.

1. Algoritmické nastavenia pre HMM



Rekurzia ($i = 1 :: N$): $v_k(i) = e_k(x_i) \max_j (a_{jk} v_j(i-1))$; $ptr_i(l) = \arg \max_j (a_{jk} v_j(i-1))$.

1. Algoritmické nastavenia pre HMM

Vyhodnotenie - Evaluation

Cieľom vyhodnotenia je určiť, ako dobre náš model dát zachytáva skutočné dáta, bez ohľadu na konkrétnu cestu.

Mnoho ciest môže vygenerovať akúkoľvek danú sekvenciu; otázkou teda je, aká je pravdepodobnosť, že sekvencia bola vytvorená týmto modelom, alebo ako dobre model zachytáva charakteristiky sekvencie? Inými slovami, je toto dobrý model? Okrem toho sa vyhodnotenie môže použiť na porovnanie rôznych modelov.

Formálna definícia problému vyhodnotenia je nasledovná:

- **Vstup:** Sekvencia pozorovaní $x = x_1 x_2 \dots x_N$ a HMM $M(Q; A; p; V; E)$.
- **Výstup:** Pravdepodobnosť, že x bolo vygenerované modelom M , sčítaná cez všetky cesty.

Vieme, že ak dostaneme HMM, môžeme vygenerovať sekvenciu dĺžky n pomocou nasledujúcich krokov:

- Z začať v stave π_1 podľa pravdepodobnosti $a_{0\pi_1}$ (získané pomocou vektora, p).
- Emitovať písmeno x_1 podľa emisnej pravdepodobnosti $e_{\pi_1}(x_1)$.
- Prejsť do stavu π_2 podľa pravdepodobnosti prechodu $a_{\pi_1|\pi_2}$.
- Pokračovať v tom, kým neemitujeme x_N .

1. Algoritmické nastavenia pre HMM

Takto môžeme emitovať akúkoľvek sekvenciu a vypočítať jej vierohodnosť. Avšak, mnoho sekvencií stavov môže emitovať to isté x , takže ako vypočítame celkovú pravdepodobnosť vygenerovania daného x cez všetky cesty? Inými slovami, naším cieľom je získať nasledujúcu pravdepodobnosť:

$$P(x|M) = P(x) = \sum_{\pi} P(x; \pi) = \sum_{\pi} P(x|\pi)P(\pi)$$

Výzvou je opäť exponenciálny počet možných ciest. **Jeden prístup by bol použiť iba Viterbiho cestu a ignorovať ostatné, keďže už vieme, ako túto cestu získať.** Jej pravdepodobnosť je však veľmi malá, keďže je to len jedna z mnohých možných ciest. Je to dobrá aproximácia len vtedy, ak má vysokú hustotu pravdepodobnosti. Správny prístup na iteratívny výpočet presného súčtu je, ako predtým, prostredníctvom dynamického programovania. **Algoritmus, ktorý to robí, je známy ako Forward algoritmus.**

1. Algoritmické nastavenia pre HMM

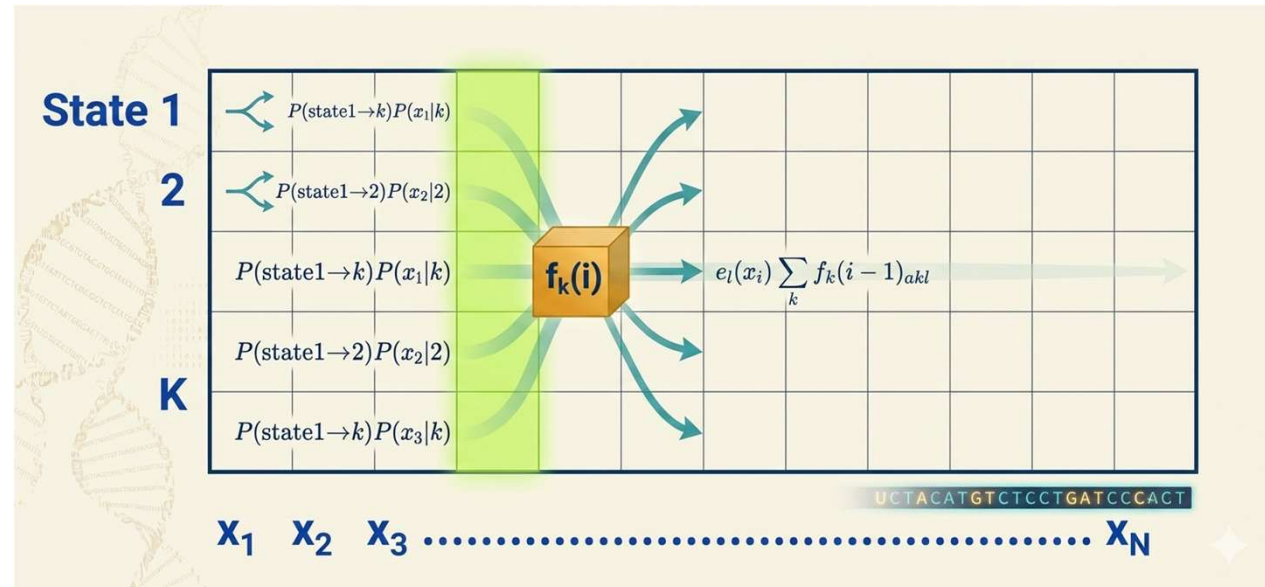
Forward algoritmus

Najprv definujeme doprednú (forward) pravdepodobnosť $f_i(i)$:

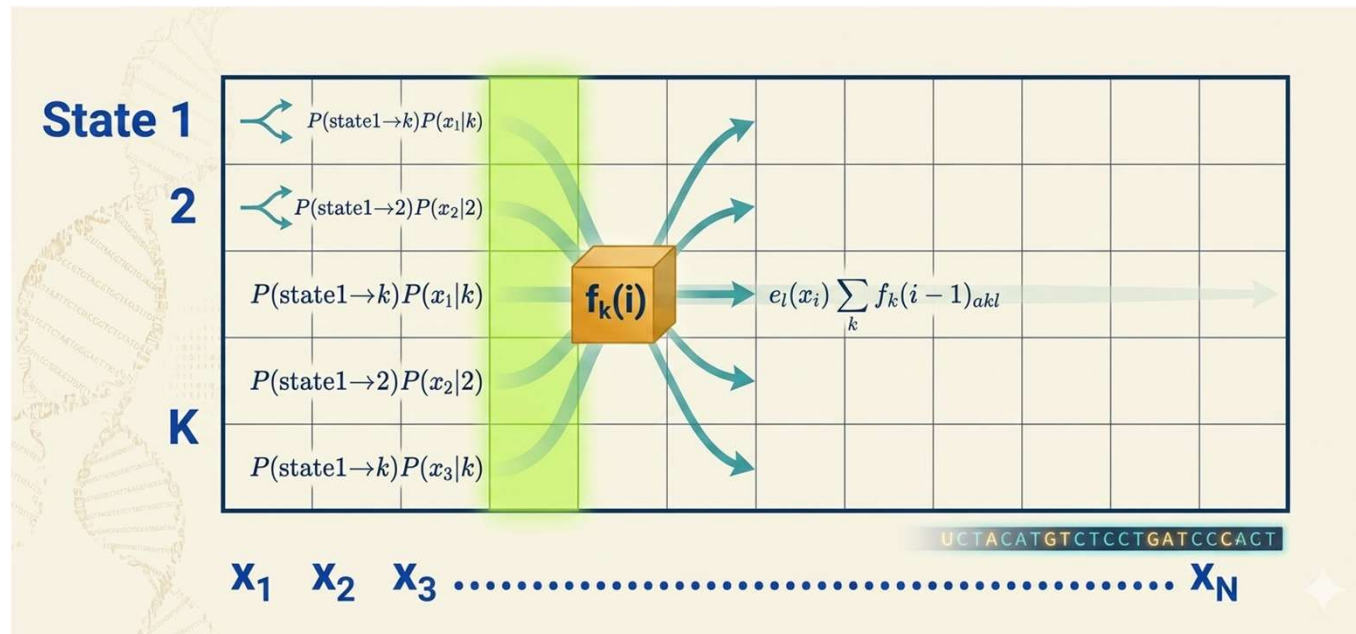
$$\begin{aligned} f_i(i) &= P(x_1 \dots x_i \mid \pi = l) \\ &= \sum_{\pi_1 \dots \pi_{i-1}} P(x_1 \dots x_{i-1} \mid (\pi_1, \dots, \pi_{i-2}, \pi_{i-1}); \pi_i = l) e_l(x_i) \\ &= \sum_k \sum_{\pi_1 \dots \pi_{i-2}} P(x_1 \dots x_{i-1} \mid (\pi_1, \dots, \pi_{i-2}); \pi_{i-1} = k) a_{kl} e_l(x_i) \\ &= \sum_k f_k(i-1) a_{kl} e_l(x_i) \\ &= e_l(x_i) \sum_k f_k(i-1) a_{kl} \end{aligned}$$

Celý algoritmus je tento:

- **Inicializácia** ($i = 0$): $f_0(0) = 1, f_k(0) = 0$ pre $k > 0$.
- **Iterácia** ($i = 1 \dots N$): $f_k(i) = e_k(x_i) \sum_j f_j(i-1) a_{jk}$.
- **Ukončenie**: $P(x, \pi^*) = \sum_k f_k(N)$



1. Algoritmické nastavenia pre HMM



Z obrázku možno vidieť, že **Forward algoritmus** je veľmi podobný **Viterbiho algoritmu**, okrem toho, že **vo Forward algoritme sa namiesto maximalizácie používa sčítanie**. Môžeme teda znovu použiť výpočty z predchádzajúceho problému vrátane penalizácie emisií, penalizácie prechodov a súčtov predchádzajúcich stavov.

Požadovaný výpočtový čas je $O(K^2N)$ a požadovaný priestor je $O(KN)$.

Nevýhodou tohto algoritmu je, že v praxi je sčítanie logaritmov zložité; preto sa namiesto toho používajú aproximácie a škálovanie pravdepodobností.

2. Viterbiho algoritmus pre mini kasino

Viterbiho algoritmus

Viterbiho algoritmus je v podstate elegantná cesta, ako sa vyhnúť "hrubej sile" (skúšaníu všetkých možných ciest).

Namiesto toho, aby sme počítali všetky kombinácie, v každom kroku si pre každý stav uložíme len **najpravdepodobnejšiu cestu, ktorá do daného stavu vedie**.

Podme si vytvoriť zjednodušený príklad, ktorý spočítame ručne.

Model: „Mini Kasíno“

Aby bol výpočet prehľadný, použijeme najskôr len 2 hody a 2 stavy.

Stavy:

- **F (Fair/Férová):** Kocka je poctivá.
- **L (Loaded/Falošná):** Kocka uprednostňuje šestky.

Parametre (zjednodušené):

- **Pravdepodobnosti prechodu (Matica A):**

$$P(F \rightarrow F) = 0,8 \text{ (Zostať pri férovosti)}$$

$$P(F \rightarrow L) = 0,2 \text{ (Prepnúť na falošnú)}$$

$$P(L \rightarrow F) = 0,2 \text{ (Prepnúť na férovú)}$$

$$P(L \rightarrow L) = 0,8 \text{ (Zostať pri falošnej)}$$

- **Emisné pravdepodobnosti (Pravdepodobnosť hodov):**

$$P(\text{hod } 1 | F) = 0,5, \quad P(\text{hod } 6 | F) = 0,1$$

$$P(\text{hod } 1 | L) = 0,1, \quad P(\text{hod } 6 | L) = 0,5$$

Pozorovaná sekvencia (x): {1, 6}

Chceme nájsť najpravdepodobnejšiu cestu $\pi^* = \{\pi_1, \pi_2\}$.

2. Viterbiho algoritmus pre mini kasino

Krok 1: Čas $t = 1$ (Prvý hod, hodnota 1)

Pre každý stav vypočítame pravdepodobnosť, že sme v ňom začali a hodili 1.

- **Pre stav F:**

$$v_F(1) = P(\text{Start } F) \cdot P(1 | F) = 0,5 \cdot 0,5 = \mathbf{0,25}$$

- **Pre stav L:**

$$v_L(1) = P(\text{Start } L) \cdot P(1 | L) = 0,5 \cdot 0,1 = \mathbf{0,05}$$

V tejto fáze máme dva "kandidátske" cesty, ktoré končia v stave F (pravdepodobnosť 0,25) alebo v stave L (pravdepodobnosť 0,05).

Krok 2: Čas $t = 2$ (Druhý hod, hodnota 6)

Teraz vypočítame pravdepodobnosti pre stavy v čase $t = 2$. Musíme zobrať do úvahy prechody z predchádzajúceho času.

Vzorec: $v_k(2) = P(6 | k) \cdot \max_j [v_j(1) \cdot A_{jk}]$

- **Výpočet pre stav F v $t = 2$:**

- Možnosť 1 (z F): $v_F(1) \cdot P(F \rightarrow F) = 0,25 \cdot 0,8 = 0,2$
- Možnosť 2 (z L): $v_L(1) \cdot P(L \rightarrow F) = 0,05 \cdot 0,2 = 0,01$
- Maximum je 0,2 (prišlo z F).
- $v_F(2) = P(6 | F) \cdot 0,2 = 0,1 \cdot 0,2 = \mathbf{0,02}$
- *Backpointer (odkiaľ sme prišli):* F

- **Výpočet pre stav L v $t = 2$:**

- Možnosť 1 (z F): $v_F(1) \cdot P(F \rightarrow L) = 0,25 \cdot 0,2 = 0,05$
- Možnosť 2 (z L): $v_L(1) \cdot P(L \rightarrow L) = 0,05 \cdot 0,8 = 0,04$
- Maximum je 0,05 (prišlo z F).
- $v_L(2) = P(6 | L) \cdot 0,05 = 0,5 \cdot 0,05 = \mathbf{0,025}$
- *Backpointer (odkiaľ sme prišli):* F

2. Viterbiho algoritmus pre mini kasino

Krok 3: Ukončenie a Spätné sledovanie (Backtracking)

1. Porovnanie koncov:

V čase $t = 2$ máme pravdepodobnosti: $v_F(2) = 0,02$ a $v_L(2) = 0,025$.

Najpravdepodobnejší stav na konci je **L** (0,025).

2. Backtracking (cesta späť):

- Začínáme v stave **L** v čase $t = 2$.
- Pozrieme sa na backpointer pre $v_L(2)$, ktorý sme si uložili: bol to stav **F**.
- Takže predchodca bol **F**.

Výsledná najpravdepodobnejšia cesta: $F \rightarrow L$

Čo to znamená?

Algoritmus nám povedal: „Najpravdepodobnejší scenár je ten, že v prvom hode bola použitá férová kocka a v druhom hode krupier prepol na falošnú kocku.“

Prečo to tak je?

- V prvom hode padla 1. Tá je oveľa pravdepodobnejšia pri férovom stave (0,5 vs 0,1).
- V druhom hode padla 6. Tá je oveľa pravdepodobnejšia pri falošnom stave (0,5 vs 0,1).

Aj keď prepnutie kocky stojí nejakú pravdepodobnosť (zníženie skóre), zisk z toho, že sme správne „uhádli“ stav pre daný hod (6-ka pri falošnej kocke), prevážil cenu tohto prepnutia.

Toto je presne to, čo Viterbiho algoritmus robí pri miliónoch nukleotidov DNA – **hľadá najlepší kompromis medzi tým, aké gény/regióny sú pravdepodobné a aké „prepnutia“ (mutácie alebo nové regióny) sú štatisticky únosné.**

2. Viterbiho algoritmus pre mini kasino

Podme rozšíriť náš príklad o tretí hod. Pozorovaná sekvencia je teraz $x = \{1, 6, 1\}$.

Naše pravidlá (parametre) zostávajú rovnaké:

- **Stavy:** F (Férová), L (Falošná)
- **Prechody (A):** $F \rightarrow F = 0.8, F \rightarrow L = 0.2, L \rightarrow F = 0.2, L \rightarrow L = 0.8$
- **Emisie (E):**
 - $P(1|F) = 0.5, P(6|F) = 0.1$
 - $P(1|L) = 0.1, P(6|L) = 0.5$
- **Štart:** $P(F) = 0.5, P(L) = 0.5$

Zopakujme si výsledky z času $t = 1$ a $t = 2$

(Tieto hodnoty už máme vypočítané):

V čase $t = 1$ (Hod '1'):

- $v_F(1) = 0.25$
- $v_L(1) = 0.05$

V čase $t = 2$ (Hod '6'):

- $v_F(2) = 0.02$ (pochádza z F)
- $v_L(2) = 0.025$ (pochádza z F)

Krok 3: Čas $t = 3$ (Tretí hod, hodnota '1')

Teraz musíme vypočítať pravdepodobnosti pre stavy F a L v treťom kroku na základe hodnôt z času $t = 2$.

Výpočet pre stav F v $t = 3$:

Hľadáme cestu, ktorá maximalizuje $v_k(2) \cdot A_{kF} \cdot P(1|F)$.

1. **Cesta z $F(2)$:** $v_F(2) \cdot P(F \rightarrow F) \cdot P(1|F) = 0.02 \cdot 0.8 \cdot 0.5 = 0.008$
 2. **Cesta z $L(2)$:** $v_L(2) \cdot P(L \rightarrow F) \cdot P(1|F) = 0.025 \cdot 0.2 \cdot 0.5 = 0.0025$
- **Maximum:** 0.008.
 - $v_F(3) = 0.008$.
 - **Backpointer (odkiaľ):** Z F .

Výpočet pre stav L v $t = 3$:

Hľadáme cestu, ktorá maximalizuje $v_k(2) \cdot A_{kL} \cdot P(1|L)$.

1. **Cesta z $F(2)$:** $v_F(2) \cdot P(F \rightarrow L) \cdot P(1|L) = 0.02 \cdot 0.2 \cdot 0.1 = 0.0004$
 2. **Cesta z $L(2)$:** $v_L(2) \cdot P(L \rightarrow L) \cdot P(1|L) = 0.025 \cdot 0.8 \cdot 0.1 = 0.002$
- **Maximum:** 0.002.
 - $v_L(3) = 0.002$.
 - **Backpointer (odkiaľ):** Z L .
-

2. Viterbiho algoritmus pre mini kasino

Krok 4: Ukončenie a Spätné sledovanie (Backtracking)

1. Porovnanie koncov:

V čase $t = 3$ máme $v_F(3) = 0.008$ a $v_L(3) = 0.002$.

Najpravdepodobnejší stav na konci je **F** (0.008).

2. Spätné sledovanie (cesta späť):

- Začínáme v stave **F** v čase $t = 3$.
- Pozrieme sa na backpointer pre $v_F(3)$: Ten nám hovorí, že cesta prišla zo stavu **F** v čase $t = 2$.
- Teraz sa pozrieme na backpointer pre $v_F(2)$: Ten nám hovorí, že cesta prišla zo stavu **F** v čase $t = 1$.

Výsledná najpravdepodobnejšia cesta: $F \rightarrow F \rightarrow F$

Analýza výsledku: Prečo vyhrala „pocitivá“ cesta?

Pozrime sa na sekvenciu hodov: **1, 6, 1**.

- Keby sme chceli využiť falošnú kocku (L) pre tú "6-ku", museli by sme prepnúť $F \rightarrow L$ a potom zasa $L \rightarrow F$ (ak chceme na konci 1-ku férovou kockou).
- Každé prepnutie stojí „poplatok“ (pravdepodobnosť prechodu 0.2).
- **Matematika Viterbiho algoritmu rozhodla:**

Cesta $F \rightarrow L \rightarrow F$ má pravdepodobnosť 0.002 (zjednodušene).

Cesta $F \rightarrow F \rightarrow F$ má pravdepodobnosť 0.008.

Hoci by falošná kocka lepšie vysvetlila tú šestku, "cena" za prepínanie kocky tam a späť je príliš vysoká vzhľadom na to, že máme len jeden hod, ktorý by falošná kocka vysvetlila lepšie. Algoritmus preto zvolil radšej "férovú" cestu, ktorá je celkovo štatisticky stabilnejšia.

2. Viterbiho algoritmus pre genomicky scanner

Model: „Genomický skener“

Tento príklad rozšírime na **3 stavy**, čo je typický model pre bioinformatickú analýzu DNA sekvencií.

Model: „Genomický skener“

Máme 3 skryté stavy:

1. **B (Background)**: Pozadie genómu (bežný výskyt nukleotidov).
2. **C (CpG Island)**: Oblasť bohatá na C a G.
3. **S (Start Motif)**: Krátka regulačná oblasť (napr. TATA box), ktorá často predchádza gény.

Parametre modelu:

- **Štart**: $P(B) = 0.4, P(C) = 0.2, P(S) = 0.4$
- **Prechody (Matica A)**:
 - $Z\ B: P(B \rightarrow B) = 0.7, P(B \rightarrow C) = 0.2, P(B \rightarrow S) = 0.1$
 - $Z\ C: P(C \rightarrow B) = 0.1, P(C \rightarrow C) = 0.8, P(C \rightarrow S) = 0.1$
 - $Z\ S: P(S \rightarrow B) = 0.4, P(S \rightarrow C) = 0.4, P(S \rightarrow S) = 0.2$
- **Emisné pravdepodobnosti (zjednodušené)**:
 - Pre $B: P(G|B) = 0.2, P(A|B) = 0.3$
 - Pre $C: P(G|C) = 0.4, P(A|C) = 0.1$
 - Pre $S: P(G|S) = 0.25, P(A|S) = 0.4$

Pozorovaná sekvencia (x): {G, A, G}

2. Viterbiho algoritmus pre genomicky scanner

Model: „Genomický skener“

Tento príklad rozšírime na **3 stavy**, čo je typický model pre bioinformatickú analýzu DNA sekvencií.

Model: „Genomický skener“

Máme 3 skryté stavy:

1. **B (Background):** Pozadie genómu (bežný výskyt nukleotidov).
2. **C (CpG Island):** Oblasť bohatá na C a G.
3. **S (Start Motif):** Krátka regulačná oblasť (napr. TATA box), ktorá často predchádza gény.

Parametre modelu:

- **Štart:** $P(B) = 0.4, P(C) = 0.2, P(S) = 0.4$
- **Prechody (Matica A):**
 - Z B: $P(B \rightarrow B) = 0.7, P(B \rightarrow C) = 0.2, P(B \rightarrow S) = 0.1$
 - Z C: $P(C \rightarrow B) = 0.1, P(C \rightarrow C) = 0.8, P(C \rightarrow S) = 0.1$
 - Z S: $P(S \rightarrow B) = 0.4, P(S \rightarrow C) = 0.4, P(S \rightarrow S) = 0.2$
- **Emisné pravdepodobnosti (zjednodušené):**
 - Pre B: $P(G|B) = 0.2, P(A|B) = 0.3$
 - Pre C: $P(G|C) = 0.4, P(A|C) = 0.1$
 - Pre S: $P(G|S) = 0.25, P(A|S) = 0.4$

Pozorovaná sekvencia (x): {G, A, G}

Krok 1: Čas $t = 1$ (Pozorovanie: 'G')

Vypočítame počiatkové pravdepodobnosti pre každý stav.

- $v_B(1) = P(B) \cdot P(G|B) = 0.4 \cdot 0.2 = \mathbf{0.08}$
- $v_C(1) = P(C) \cdot P(G|C) = 0.2 \cdot 0.4 = \mathbf{0.08}$
- $v_S(1) = P(S) \cdot P(G|S) = 0.4 \cdot 0.25 = \mathbf{0.10}$

Krok 2: Čas $t = 2$ (Pozorovanie: 'A')

Teraz pre každý stav ($k \in \{B, C, S\}$) hľadáme najlepšieho predchodcu.

Vzorec: $v_k(2) = P(A|k) \cdot \max_j [v_j(1) \cdot A_{jk}]$

- **Pre stav B:**
 - Max: $\max(0.08 \cdot 0.7, 0.08 \cdot 0.1, 0.10 \cdot 0.4) = \max(0.056, 0.008, 0.04) = 0.056$ (z B)
 - $v_B(2) = P(A|B) \cdot 0.056 = 0.3 \cdot 0.056 = \mathbf{0.0168}$
- **Pre stav C:**
 - Max: $\max(0.08 \cdot 0.2, 0.08 \cdot 0.8, 0.10 \cdot 0.4) = \max(0.016, 0.064, 0.04) = 0.064$ (z C)
 - $v_C(2) = P(A|C) \cdot 0.064 = 0.1 \cdot 0.064 = \mathbf{0.0064}$
- **Pre stav S:**
 - Max: $\max(0.08 \cdot 0.1, 0.08 \cdot 0.1, 0.10 \cdot 0.2) = \max(0.008, 0.008, 0.02) = 0.02$ (z S)
 - $v_S(2) = P(A|S) \cdot 0.02 = 0.4 \cdot 0.02 = \mathbf{0.0080}$

2. Viterbiho algoritmus pre genomicky scanner

Krok 3: Čas $t = 3$ (Pozorovanie: 'G')

Hľadáme najlepšieho predchodcu z času $t = 2$.

Vzorec: $v_k(3) = P(G | k) \cdot \max_j [v_j(2) \cdot A_{jk}]$

- **Pre stav B:**

- Max:
 $\max(v_B(2) \cdot 0.7, v_C(2) \cdot 0.1, v_S(2) \cdot 0.4) = \max(0.01176, 0.00064, 0.0032) = 0.01176$
(z B)
- $v_B(3) = P(G | B) \cdot 0.01176 = 0.2 \cdot 0.01176 = \mathbf{0.002352}$

- **Pre stav C:**

- Max:
 $\max(v_B(2) \cdot 0.2, v_C(2) \cdot 0.8, v_S(2) \cdot 0.4) = \max(0.00336, 0.00512, 0.0032) = 0.00512$
(z C)
- $v_C(3) = P(G | C) \cdot 0.00512 = 0.4 \cdot 0.00512 = \mathbf{0.002048}$

- **Pre stav S:**

- Max:
 $\max(v_B(2) \cdot 0.1, v_C(2) \cdot 0.1, v_S(2) \cdot 0.2) = \max(0.00168, 0.00064, 0.0016) = 0.00168$
(z B)
- $v_S(3) = P(G | S) \cdot 0.00168 = 0.25 \cdot 0.00168 = \mathbf{0.00042}$

Krok 4: Ukončenie a Spätné sledovanie (Backtracking)

1. Porovnanie koncov:

- $v_B(3) = 0.002352$
- $v_C(3) = 0.002048$
- $v_S(3) = 0.00042$
- Najvyšší je stav **B** (Background).

2. Backtracking:

- Stav v $t = 3$ je **B**.
- Pozrieme sa, odkiaľ prišlo $v_B(3)$: Prišlo zo stavu **B** v čase $t = 2$.
- Pozrieme sa, odkiaľ prišlo $v_B(2)$: Prišlo zo stavu **B** v čase $t = 1$.

Výsledná najpravdepodobnejšia cesta: $B \rightarrow B \rightarrow B$

Prečo algoritmus vybral "B"?

Aj keď v sekvencii boli G-čka (ktoré má rada C) a A-čko, model je nastavený tak, že **zotrvanie v stave Background (0.7)** je veľmi pravdepodobné. Prepnutie do CpG ostrova (C) alebo regulačnej oblasti (S) "stojí" príliš veľa pravdepodobnosti vzhľadom na krátku sekvenciu. Keby sekvencia pokračovala ďalej (napr. ďalšie 3x 'G'), algoritmus by s najväčšou pravdepodobnosťou "prepol" do stavu C.

Tento ručný výpočet vám ukázal, že **Viterbi nie je len o maximách emisií, ale o rovnováhe medzi emisiami a pravdepodobnosťou prechodu.**

2. Viterbiho algoritmus pre genomicky scanner

Model: „Genomický skener II“

Ukážeme si, ako citlivo reaguje algoritmus na zmenu parametrov (tzv. "naladenie modelu").

Zmeníme maticu prechodov tak, aby náš model „preferoval“ CpG ostrovy (stav C).

- Zvýšime pravdepodobnosť prechodu z **B** → **C** (zo 0.2 na **0.4**).
- Zvýšime pravdepodobnosť zotrvania v **C** → **C** (z 0.8 na **0.85**).

Nové parametre prechodov (Matica A):

- Z **B**: $P(B \rightarrow B) = 0.5, P(B \rightarrow C) = 0.4, P(B \rightarrow S) = 0.1$
- Z **C**: $P(C \rightarrow B) = 0.1, P(C \rightarrow C) = 0.85, P(C \rightarrow S) = 0.05$
- Z **S**: $P(S \rightarrow B) = 0.4, P(S \rightarrow C) = 0.4, P(S \rightarrow S) = 0.2$

(Emisné pravdepodobnosti zostávajú rovnaké ako minule: $B=[0.2G, 0.3A]$, $C=[0.4G, 0.1A]$, $S=[0.25G, 0.4A]$)

Krok 1: Čas $t = 1$ (Pozorovanie: 'G')

Začiatkové pravdepodobnosti sú rovnaké:

- $v_B(1) = 0.08$
- $v_C(1) = 0.08$
- $v_S(1) = 0.10$

Krok 2: Čas $t = 2$ (Pozorovanie: 'A')

Teraz hľadáme nové maximá s upravenými prechodmi.

- **Pre stav B:**
 - Max: $\max(0.08 \cdot 0.5, 0.08 \cdot 0.1, 0.1 \cdot 0.4) = \max(0.04, 0.008, 0.04) = 0.04$
 - $v_B(2) = 0.3 \cdot 0.04 = \mathbf{0.012}$ (cesta B alebo S → B)
 - **Pre stav C:**
 - Max: $\max(0.08 \cdot 0.4, 0.08 \cdot 0.85, 0.1 \cdot 0.4) = \max(0.032, \mathbf{0.068}, 0.04) = 0.068$
 - $v_C(2) = 0.1 \cdot 0.068 = \mathbf{0.0068}$ (cesta C → C)
 - **Pre stav S:**
 - Max: $\max(0.08 \cdot 0.1, 0.08 \cdot 0.05, 0.1 \cdot 0.2) = 0.02$
 - $v_S(2) = 0.4 \cdot 0.02 = \mathbf{0.008}$
-

2. Viterbiho algoritmus pre genomicky scanner

Krok 3: Čas $t = 3$ (Pozorovanie: 'G')

Tu sa prejaví zmena modelu.

- **Pre stav B:**

- Max: $\max(v_B(2) \cdot 0.5, v_C(2) \cdot 0.1, v_S(2) \cdot 0.4) = \max(0.006, 0.00068, 0.0032) = 0.006$
- $v_B(3) = 0.2 \cdot 0.006 = \mathbf{0.0012}$

- **Pre stav C:**

- Max:
 $\max(v_B(2) \cdot 0.4, v_C(2) \cdot 0.85, v_S(2) \cdot 0.4) = \max(0.0048, \mathbf{0.00578}, 0.0032) = 0.00578$
- $v_C(3) = 0.4 \cdot 0.00578 = \mathbf{0.002312}$

- **Pre stav S:**

- Max: $\max(v_B(2) \cdot 0.1, v_C(2) \cdot 0.05, v_S(2) \cdot 0.2) = 0.0016$
- $v_S(3) = 0.25 \cdot 0.0016 = \mathbf{0.0004}$

Krok 4: Výsledok (Backtracking)

1. Porovnanie koncov:

- $v_B(3) = 0.0012$
- $v_C(3) = 0.002312$
- $v_S(3) = 0.0004$

Vítaz je stav C.

2. Backtracking:

- Stav v $t = 3$ je **C**.
- Z $v_C(3)$ vieme, že cesta prišla z $v_C(2)$ (ktoré bolo C).
- Z $v_C(2)$ vieme, že cesta prišla z $v_C(1)$ (ktoré bolo C).

Výsledná cesta: C → C → C

Čo sa stalo?

Zmenou parametrov sme **znížili bariéru vstupu** do CpG ostrova. Keď algoritmus videl v modeli, že je "lacnejšie" zostať v stave C (pretože $C \rightarrow C = 0.85$), model sa začal správať ako "detektor ostrovov" a celú sekvenciu $\{G, A, G\}$ interpretoval ako súčasť CpG ostrova, hoci tam bolo aj 'A'.

Toto je kľúčový moment: **Výsledok Viterbiho algoritmu závisí od správneho nastavenia pravdepodobností prechodov**. Ak model nastavíte príliš "prísne", bude všetko považovať za pozadie. Ak ho nastavíte príliš "benevolentne", nájde gény aj tam, kde nie sú. V bioinformatike je ladenie týchto pravdepodobností (training/learning) práve to, čomu sa hovorí učenie modelu.

Je tento posun v správaní algoritmu zrozumiteľný?

2. Forward algoritmus pre genomicky scanner

Forward algoritmus

Forward algoritmus je **súčet pravdepodobností všetkých možných ciest**. Zatiaľ čo Viterbi hľadá tú jednu „najlepšiu“ cestu (maximalizuje), Forward algoritmus sa pýta: „Aká je celková pravdepodobnosť, že tento model vygeneroval túto sekvenciu, keď zoberiem do úvahy všetky kombinácie stavov?“

V matematickom zápise je jediný rozdiel: **namiesto operácie max používame sum (súčet)**.

Použijeme úplne rovnaké parametre (stavy B, C, S a sekvenciu $\{G, A, G\}$) ako v predchádzajúcom príklade, aby ste videli ten rozdiel.

Pravidlá (Pripomenutie)

- **Prechody (A):**
 - $Z B \rightarrow \{B:0.5, C:0.4, S:0.1\}$
 - $Z C \rightarrow \{B:0.1, C:0.85, S:0.05\}$
 - $Z S \rightarrow \{B:0.4, C:0.4, S:0.2\}$
- **Emisie (E):** $B(G:0.2, A:0.3), C(G:0.4, A:0.1), S(G:0.25, A:0.4)$

Krok 1: Čas $t = 1$ (Pozorovanie 'G')

Inicializácia je rovnaká ako pri Viterbi:

- $f_B(1) = 0.4 \cdot 0.2 = 0.08$
- $f_C(1) = 0.2 \cdot 0.4 = 0.08$
- $f_S(1) = 0.4 \cdot 0.25 = 0.1$

Krok 2: Čas $t = 2$ (Pozorovanie 'A')

Teraz používame **sumu**. Pre každý stav k vypočítame:

$$f_k(2) = P(A|k) \cdot \sum_j [f_j(1) \cdot A_{jk}]$$

- **Pre stav B:**
 - Suma predchodcov:
 $(0.08 \cdot 0.5 + 0.08 \cdot 0.1 + 0.1 \cdot 0.4) = (0.04 + 0.008 + 0.04) = 0.088$
 - $f_B(2) = 0.3 \cdot 0.088 = \mathbf{0.0264}$
- **Pre stav C:**
 - Suma predchodcov:
 $(0.08 \cdot 0.4 + 0.08 \cdot 0.85 + 0.1 \cdot 0.4) = (0.032 + 0.068 + 0.04) = 0.14$
 - $f_C(2) = 0.1 \cdot 0.14 = \mathbf{0.014}$
- **Pre stav S:**
 - Suma predchodcov:
 $(0.08 \cdot 0.1 + 0.08 \cdot 0.05 + 0.1 \cdot 0.2) = (0.008 + 0.004 + 0.02) = 0.032$
 - $f_S(2) = 0.4 \cdot 0.032 = \mathbf{0.0128}$

2. Forward algoritmus pre genomicky scanner

Krok 3: Čas $t = 3$ (Pozorovanie 'G')

Opäť sčítavame pravdepodobnosti zo všetkých možných ciest z $t = 2$.

- **Pre stav B:**

- Suma predchodcov:
 $(0.0264 \cdot 0.5 + 0.014 \cdot 0.1 + 0.0128 \cdot 0.4) = (0.0132 + 0.0014 + 0.00512) = 0.01972$
- $f_B(3) = 0.2 \cdot 0.01972 = \mathbf{0.003944}$

- **Pre stav C:**

- Suma predchodcov:
 $(0.0264 \cdot 0.4 + 0.014 \cdot 0.85 + 0.0128 \cdot 0.4) = (0.01056 + 0.0119 + 0.00512) = 0.02758$
- $f_C(3) = 0.4 \cdot 0.02758 = \mathbf{0.011032}$

- **Pre stav S:**

- Suma predchodcov:
 $(0.0264 \cdot 0.1 + 0.014 \cdot 0.05 + 0.0128 \cdot 0.2) = (0.00264 + 0.0007 + 0.00256) = 0.0059$
- $f_S(3) = 0.25 \cdot 0.0059 = \mathbf{0.001475}$

Krok 4: Ukončenie (Celková pravdepodobnosť)

Na konci už nehľadáme maximum, ale **sčítame všetky hodnoty v poslednom stĺpci**.

$$P(x) = f_B(3) + f_C(3) + f_S(3)$$

$$P(x) = 0.003944 + 0.011032 + 0.001475 = \mathbf{0.016451}$$

Čo nám to hovorí?

- **Viterbi** nám dal odpoveď: "Najpravdepodobnejšia cesta je C-C-C s pravdepodobnosťou 0.002312."
- **Forward** nám hovorí: "Všetky možné cesty, ktoré môžu vygenerovať túto sekvenciu, majú spolu pravdepodobnosť 0.016451."

Prečo je to dôležité?

Ak si predstavíte, že máte dva rôzne modely (napr. jeden pre "normálnu DNA" a jeden pre "vírusovú DNA"), pomocou Forward algoritmu viete pre danú sekvenciu vypočítať **obe** celkové pravdepodobnosti. **Ten model, ktorý vám vráti vyššie číslo, je ten, ktorý lepšie "vysvetľuje" danú sekvenciu.**

"hľadanie najlepšej cesty" (Viterbi) vs "sčítanie všetkých možností" (Forward)

3. Porovnanie

To, že sa zdá ťažké rozlíšiť Viterbi a Forward, je úplne prirodzené.

Obe používajú rovnaké „stavebné kamene“ (prechody a emisie), ale majú úplne odlišný **cieľ**.

1. Viterbi algoritmus = Detektív („Čo sa vlastne stalo?“)

Predstav si, že nájdeš denník (pozorované dáta), ale nevieš, kto ho písal. Viterbi je detektív, ktorý hľadá **jeden najpravdepodobnejší príbeh**.

- **Tvoja otázka:** „Keď vidím tieto stopy (emisie), aký bol najpravdepodobnejší sled udalostí (skryté stavy), ktoré k nim viedli?“
- **Čo robí:** Skúša nájsť jednu konkrétnu „cestu“ cez stavy, ktorá vysvetľuje tvoje dáta najlepšie.
- **Výstup:** Dostanete **zoznam stavov**. (Např. „Človek bol najprv smutný, potom veselý, potom smutný.“)
- **Prečo je to dôležité pre projekt:** Toto je kľúč k **vysvetliteľnosti (explainability)**. Viterbi nám povie: „Pozri, model označil tento úsek DNA za vírusový, pretože to bola štatisticky najpravdepodobnejšia cesta.“ Vieš ukázať prstom na konkrétne miesto a povedať, prečo sa model tak rozhodol.

2. Forward algoritmus = Štatistik („Je tento model dobrý?“)

Štatistik sa nestará o to, čo sa presne stalo. **On sa pýta na celkovú „vierohodnosť“.**

- **Tvoja otázka:** „Mám tento model (např. model pre vírusy). Keď mu dám túto DNA sekvenciu, ako veľmi mu 'sedí'? Je tento model schopný vygenerovať takéto dáta?“
- **Čo robí:** Nerieši, aká cesta je najlepšia. Sčíta pravdepodobnosti úplne všetkých ciest, ktoré by mohli viesť k vašim dátam.
- **Výstup:** Dostanete **jedno číslo** (pravdepodobnosť). (Např. „Pravdepodobnosť, že tento model vygeneroval túto DNA, je 0.0001 %.“)
- **Prečo je to dôležité:** Slúži na **hodnotenie kvality (Evaluation)**. Ak máš dva modely (např. „Ľudský genóm“ vs „Vírus“), týmto zistíte, ktorému z nich tvoje dáta „patria“ viac.

3. Porovnanie

Vlastnosť	Viterbi	Forward
Hlavný cieľ	Nájdenie najlepšej cesty .	Nájdenie celkovej pravdepodobnosti .
Operácia	Používa MAX (hľadá vrchol).	Používa SUM (sčíta všetko).
Výsledok	Sekvencia stavov (napr. "Vírus-Vírus-Genóm").	Jediné číslo (skóre modelu).
Využitie	Anotácia (chcem vedieť, kde je gén).	Validácia (chcem vedieť, či je model správny).

Kódovanie pamäte v HMM - Detekcia CpG ostrovčekov

Môžeme do nášho modelu zahrnúť pamäť?

Odpoveď na túto otázku znie – Áno, môžeme!

Ale ako? **Pripomeňme si, že Markovove modely sú bezpamäťové.** Inými slovami, všetka pamäť modelu je obsiahnutá v stavoch. **Takže, aby sme mohli uložiť dodatočné informácie, musíme zvýšiť počet stavov.**

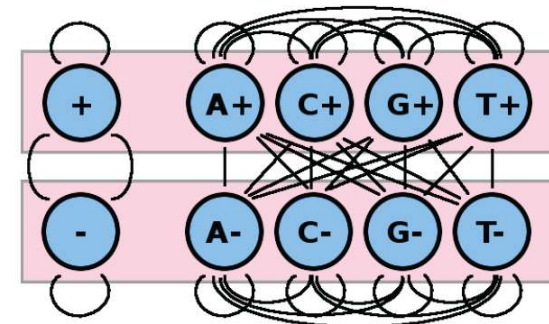
Teraz sa pozrime späť na biologický príklad. V našom modeli emisie stavov záviseli iba od aktuálneho stavu. A aktuálny stav kodoval iba jeden nukleotid. Čo však ak chceme, aby náš model počítal frekvencie dinukleotidov (pre CpG ostrovčeky¹), alebo trinukleotidové frekvencie (pre kodóny), alebo dikodónové frekvencie zahrňajúce šesť nukleotidov?

CpG ostrovčeky sú definované ako oblasti v genóme, ktoré sú obohatené o páry nukleotidov C a G na tom istom vlákne.

- Typicky, keď je tento dinukleotid prítomný v genóme, metyluje sa a keď dôjde k deaminácii cytozínu, ako sa to deje pri určitej základnej frekvencii, stane sa tymínom, ďalším prirodzeným nukleotidom, a preto ho bunka nemôže tak ľahko rozpoznať ako mutáciu, čo spôsobuje mutáciu C na T.
- Táto zvýšená frekvencia mutácií na CpG ostrovčekoch v priebehu evolučného času vyčerpáva CpG ostrovčeky a robí ich relatívne vzácnymi.
- Keďže metylácia môže nastať na ktoromkoľvek vlákne, CpG zvyčajne mutujú na TpG alebo CpA.
- Ak sú však umiestnené v aktívnom promótore, **metylácia je potlačená a CpG dinukleotidy sú schopné pretrvávať.**
- **Podobne sú CpG v oblastiach dôležitých pre funkciu bunky konzervované vďaka evolučnému tlaku.** V dôsledku toho môže detekcia CpG ostrovčekov zvýrazniť oblasti promótora, iné transkripčne aktívne oblasti alebo miesta purifikačnej selekcie v genóme.

Vedeli ste?

CpG je skratka pre [C]ytozín - [p]fosfátový reťazec - [G]uanín. Písmeno „p“ znamená, že hovoríme o tom istom vlákne dvojitej špirály, a nie o bázevom páre GC, ktorý sa nachádza naprieč špirálou.



4. Špeciálne otázky

Vzhľadom na ich biologický význam sú CpG ostrovčeky hlavnými kandidátmi na modelovanie.

- Spočiatku sa možno pokúsiť identifikovať tieto ostrovčeky skenovaním genómu a hľadať v nich fixné intervaly bohaté na GC. Účinnosť tohto prístupu je ohrozená výberom vhodnej veľkosti okna; zatiaľ čo príliš malé okno nemusí zachytiť celý konkrétny CpG ostrovček, príliš veľké okno by viedlo k vynechaniu mnohých menších, ale skutočných CpG ostrovčekov.
- Skúmanie genómu na základe kodónov tiež vedie k ťažkostiam, pretože páry CpG nemusia nevyhnutne kódovať aminokyseliny, a preto nemusia ležať v jednom kodóne.
- Namiesto toho sú HMM oveľa vhodnejšie na modelovanie tohto scenára, pretože, ako čoskoro uvidíme v časti o unsupervised učení, HMM môžu prispôbiť svoje základné parametre tak, aby maximalizovali svoju pravdepodobnosť.

Nie všetky HMM sú však na túto konkrétnu úlohu vhodné.

Model HMM, ktorý zohľadňuje iba frekvencie jednotlivých nukleotidov C a G, nedokáže zachytiť povahu CpG ostrovčekov.

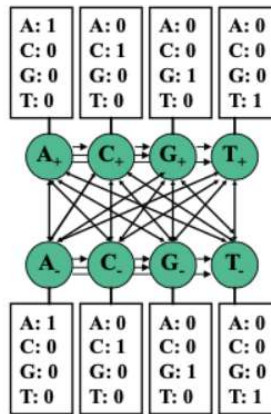
Uvažujme jeden takýto HMM s dvoma nasledujúcimi skrytými stavmi:

- Stav „+“ predstavuje ostrovy CpG
- Stav „-“: predstavuje neostrovné oblasti

Každý z týchto dvoch stavov potom s určitou pravdepodobnosťou emituje bázy A, C, G a T. Hoci ostrovčeky CpG v tomto modeli môžu byť obohatené o bázy C a G zvýšením ich príslušných emisných pravdepodobností, tento model nedokáže zachytiť skutočnosť, že C a G sa vyskytujú prevažne v pároch.

4. Špeciálne otázky

- **Vzhľadom na Markovovu vlastnosť, ktorá riadi HMM, jediné informácie dostupné v každom časovom kroku musia byť obsiahnuté v aktuálnom stave.**
- **Preto na kódovanie pamäte v rámci Markovovho reťazca musíme rozšíriť stavový priestor.** Na to je možné jednotlivé stavy „+“ a „-“ nahradiť 4 stavmi „+“ a 4 stavmi „-“: A+, C+, G+, T+, A-, C-, G-, T-



Konkrétne existujú 2 spôsoby modelovania a táto voľba bude mať za následok rôzne pravdepodobnosti emisií:

- Jeden model naznačuje, že stav A+ napríklad znamená, že sa momentálne nachádzame v CpG ostrove a predchádzajúci znak bol A. Pravdepodobnosti emisií tu budú niest väčšinu informácií a prechody budú dosť degenerované.
- Iný model naznačuje, že stav A+ napríklad znamená, že sa momentálne nachádzame v CpG ostrove a aktuálny znak je A. Pravdepodobnosť emisie tu bude 1 pre A a 0 pre všetky ostatné písmená a pravdepodobnosti prechodu budú niest väčšinu informácií v modeli a emisie budú dosť degenerované. Odteraz budeme predpokladať tento model.

Vedeli ste?

Počet prechodov je druhá mocnina počtu stavov. To dáva hrubú predstavu o tom, ako sa zvyšuje „pamäť“ (a teda aj stavy) HMM.

Pamäť tohto systému je odvodená zo skutočnosti, že každý stav môže emitovať iba jeden znak, a preto si svoj emitovaný znak „pamätá“.

4. Špeciálne otázky

Vedeli ste?

- **Počet prechodov je druhá mocnina počtu stavov.** To dáva hrubú predstavu o tom, ako sa zvyšuje „pamäť“ (a teda aj stavy) HMM.
- Pamäť tohto systému je odvodená zo skutočnosti, že každý stav môže emitovať iba jeden znak, a preto si svoj emitovaný znak „pamätá“.
- Okrem toho je dinukleotidová povaha CpG ostrovčekov začlenená do prechodových matíc. Najmä frekvencia prechodov zo stavov C+ do G+ je výrazne vyššia ako zo stavov C- do G-, čo dokazuje, že tieto páry sa v rámci ostrovčekov vyskytujú častejšie.

Otázka: Keďže každý stav vydáva iba jeden znak, môžeme povedať, že sa to redukuje na Markovov reťazec namiesto HMM?

Nie. Aj keď emisie označujú písmeno skrytého stavu, neindikujú, či je stav CpG ostrovom alebo nie: stav A- aj A+ emitujú iba pozorovateľný stav A.

Otázka: Ako môžeme začleniť naše znalosti o systéme pri tréňovaní HMM modelov, napr. niektoré pravdepodobnosti emisií 0 v prípade detekcie CpG ostrova?

Mohli by sme buď vnútiť modelu naše znalosti nastavením niektorých parametrov a ponechať iné, aby sa menili, alebo by sme mohli nechať HMM voľne pracovať s modelom a nechať ho objaviť tieto vzťahy. V skutočnosti existujú dokonca metódy, ktoré model zjednodušujú tým, že vynútia podmnožinu parametrov na 0, ale umožnia HMM vybrať si, ktorú podmnožinu.

Vzhľadom na vyššie uvedený rámec môžeme použiť posteriórne dekódovanie na analýzu každej bázy v genóme a určenie, či je s najväčšou pravdepodobnosťou súčasťou CpG ostrova alebo nie. Ale po zostavení rozšíreného HMM modelu, ako môžeme overiť, či je v skutočnosti lepší ako model s jedným nukleotidom? Predtým sme demonštrovali, že algoritmus dopredu alebo dozadu možno použiť na výpočet $P(x)$ pre daný model. Ak je pravdepodobnosť nášho súboru údajov vyššia pri druhom modeli ako pri prvom modeli, s najväčšou pravdepodobnosťou efektívnejšie zachytáva základné správanie.

4. Špeciálne otázky

Existuje však jedno riziko spojené s komplikáciou modelu, a to preusporiadanie.

- Zvýšenie počtu parametrov pre HMM zvyšuje pravdepodobnosť preusporiadania dát zo strany HMM a znižuje presnosť zachytenia základného správania.
- Bežným riešením v strojovom učení je použitie **regularizácie**, čo v podstate znamená použitie menšieho počtu parametrov. V tomto prípade je možné znížiť počet parametrov, ktoré sa majú naučiť, obmedzením všetkých pravdepodobností prechodu +/- na rovnakú hodnotu a všetkých pravdepodobností prechodu -/+ na rovnakú hodnotu, pretože prechody tam a späť z + a - stavov sú to, čo nás zaujíma pri modelovaní, a skutočné bázy, kde k prechodu došlo, nie sú pre náš model až také dôležité. Preto sa pre tento obmedzený model musíme naučiť menej parametrov, čo vedie k jednoduchšiemu modelu a môže pomôcť vyhnúť sa preusporiadaniu.

Otázka: Existujú aj iné spôsoby kódovania pamäte pre detekciu CpG ostrovčekov?

Odpoveď: Ďalšie nápady, s ktorými by sa dalo experimentovať, zahŕňajú

- vyhľadávajúte dinukleotidy a nájdite spôsob, ako sa vysporiadať s prekrývaním.
- pridajte špeciálny stav, ktorý prechádza z C do G.

4. Špeciálne otázky

Conditional random fields

Podmienené náhodné polia (CRF) modelujú diskriminačný, neorientovaný pravdepodobnostný grafický model, ktorý sa používa ako alternatíva k HMM.

Používa sa na kódovanie známych vzťahov medzi pozorovaniami a na konštrukciu konzistentných interpretácií.

Často sa používa na označovanie alebo syntaktickú analýzu (parsing) sekvenčných dát. Široko sa využíva pri vyhľadávaní génov.

Nasledujúce zdroje môžu byť užitočné, ak sa chcete dozvedieť viac o CRF:

- Prednáška o podmienených náhodných poliach z kurzu Probabilistic Graphical Models:
<https://www.coursera.org/specializations/probabilistic-graphical-models#courses> .
- **Biological Entity Recognition with Conditional Random Fields**
<https://pmc.ncbi.nlm.nih.gov/articles/PMC2656029/>
<https://people.cs.umass.edu/~mccallum/papers/crfgene.pdf>
- **An Introduction to Conditional Random Fields** <https://people.cs.umass.edu/~mccallum/papers/crf-tutorial.pdf>

4. Špeciálne otázky

Nástroje na implementáciu HMM

Skryté Markovove modely a súvisiace algoritmy predstavené v tejto časti boli implementované v niekoľkých open-source balíkoch, ktoré zjednodušujú tvorbu, tréning a analýzu HMM. Nižšie je ukážka dostupných nástrojov.

- Balík 'HMM' pre R vám umožňuje inicializovať HMM a spustiť niekoľko typov analýz. Implementované sú napríklad funkcie na výpočet dopredných (forward), spätných (backward) a posteriorných pravdepodobností, ako aj model "nepoctivého kasína", ako je prezentovaný v tejto časti. Balík je dostupný v CRAN; viac informácií nájdete v dokumentácii: <https://cran.r-project.org/web/packages/HMM/HMM.pdf>
- V Pythone sú HMM implementované v balíku hmmlearn, ktorý obsahuje všetky algoritmy diskutované v tejto aj nasledujúcej prednáške. Kód a dokumentácia sú dostupné tu: <https://github.com/hmmlearn/hmmlearn>

4. Špeciálne otázky

Čo sme sa naučili?

V tejto časti sme pokryli nasledujúci hlavný obsah:

- Rozumieme, ako modelovať sekvenčné dáta rozpoznáním typu sekvencie: genomické, orálne, verbálne, vizuálne atď. (nielen či sú podobné, ale či sú rovnakej triedy)
- **Predstavili sme motiváciu za skrytými Markovovými modelmi v našej analýze anotácie genómu.**
- Formalizovali sme Markovove reťazce a HMM na príklade predpovede počasia.
- Získali sme predstavu o tom, ako aplikovať HMM na reálne dáta pri pohľade na problémy "nepoctivého kasína" a CG-bohatých oblastí.
- Systematicky sme predstavili **algoritmické nastavenia HMM** a podrobne sme sa venovali trom z nich:
 - Skórovanie: **skórovanie cez jedinú cestu**
 - Skórovanie: **skórovanie cez všetky cesty**
 - Dekódovanie: **Viterbiho kódovanie pri určovaní najpravdepodobnejšej cesty**
- Konkrétnejšie vieme, že pri danom x môžeme vypočítať y :
 - Spustenie modelu: pri danom modeli môžeme generovať sekvenciu "typu"
 - Vyhodnotenie: pri danom modeli, emisiách a stavoch môžeme zistiť pravdepodobnosť
 - Viterbi: pri danom modeli a emisiách môžeme určiť optimálnu cestu
 - Forward: pri danom modeli a emisiách môžeme nájsť celkovú pravdepodobnosť cez všetky cesty

Posterior decoding

- Táto prednáška bude diskutovať o posteriornom dekodovaní (posterior decoding), **algoritme, ktorý opäť odvodí sekvenciu skrytých stavov π , ktorá maximalizuje odlišnú metriku.**
- Konkrétne, nájde najpravdepodobnejší stav na každej pozícii naprieč všetkými možnými cestami, a to pomocou dopredného aj spätného algoritmu.
- Následne ukážeme, ako zakódovať „pamäť“ do Markovovho reťazca pridaním ďalších stavov, aby sme prehľadali genóm pre stavy uchovávajúce dinukleotidy.
- Potom budeme diskutovať o tom, ako použiť odhad parametrov metódou maximálnej vierohodnosti (Maximum Likelihood) pre supervizované učenie s označenou množinou dát.
- Stručne uvidíme, ako použiť Viterbiho učenie pre nesupervizovaný odhad parametrov neoznačenej množiny dát.
- Nakoniec sa naučíme, ako použiť algoritmus očakávania a maximalizácie (Expectation Maximization – EM) pre nesupervizovaný odhad parametrov neoznačenej množiny dát, kde špecifický algoritmus pre HMM je známy ako Baum-Welchov algoritmus.

Posteriórne dekódovanie (Posterior Decoding)

Motivácia

Hoci Viterbiho algoritmus dekódovania poskytuje jeden spôsob odhadu skrytých stavov podkladajúcich sekvenciu pozorovaných znakov, ďalší platný spôsob inferencie (odvodzovania) poskytuje **posteriórne dekódovanie**.

Posteriórne dekódovanie poskytuje najpravdepodobnejší stav v akomkoľvek časovom bode.

Aby sme získali určitú intuíciu pre posteriórne dekódovanie, pozrime sa, ako sa aplikuje na **situáciu, v ktorej nepoctivé kasíno strieda férovú a cinknutú kocku**. Predpokladajme, že vstúpime do kasína s vedomím, že nefér (cinknutá) kocka sa používa 60 percent času. S touto znalosťou a bez akýchkoľvek hodov kockou je náš najlepší odhad pre aktuálnu kocku zjavne tá cinknutá. Po jednom hode je pravdepodobnosť, že bola použitá cinknutá kocka, daná vzorcom:

$$P(\text{kocka} = \text{cinknutá} \mid \text{hod} = k) = \frac{P(\text{kocka} = \text{cinknutá}) \cdot P(\text{hod} = k \mid \text{kocka} = \text{cinknutá})}{P(\text{hod} = k)}$$

Ak by sme namiesto toho pozorovali sekvenciu N hodov kockou, ako by sme vykonali podobný druh inferencie? Tým, že **dovolíme informáciám prúdiť medzi N hodmi a ovplyvňovať pravdepodobnosť každého stavu**, je posteriórne dekódovanie prirodzeným rozšírením vyššie uvedenej inferencie na sekvenciu ľubovoľnej dĺžky.

5. Posterior Decoding

Ak by sme namiesto toho pozorovali sekvenciu N hodov kockou, ako by sme vykonali podobný druh inferencie? Tým, že **dovolíme informáciám prúdiť medzi N hodmi a ovplyvňovať pravdepodobnosť každého stavu, je posteriórne dekódovanie prirodzeným rozšírením vyššie uvedenej inferencie na sekvenciu ľubovoľnej dĺžky.**

- Formálnejšie povedané, namiesto identifikácie jednej cesty maximálnej vierohodnosti, posteriórne dekódovanie uvažuje o pravdepodobnosti, že akákoľvek cesta sa nachádza v stave k v čase t pri daných všetkých pozorovaných znakoch, t. j. $P(\pi_t = k \mid x_1; \dots; x_n)$.
- Stav, ktorý maximalizuje túto pravdepodobnosť pre daný čas, sa potom považuje za najpravdepodobnejší stav v tomto bode.

Je dôležité poznamenať, že okrem **informácií prúdiacich dopredu** na určenie najpravdepodobnejšieho stavu v bode, môžu **informácie prúdiť aj spätne od konca sekvencie k tomuto stavu, aby sa zvýšila alebo znížila vierohodnosť každého stavu v danom bode.** Je to čiastočne prirodzený dôsledok reverzibility Bayesovho pravidla: naše pravdepodobnosti sa pri pozorovaní ďalších údajov menia z apriórnych pravdepodobností na aposteriórne (posteriórne) pravdepodobnosti.

5. Posterior Decoding

Aby sme to objasnili, predstavte si opäť príklad s kasínom. Ako bolo uvedené skôr, bez pozorovania akýchkoľvek hodov je stav0 s najväčšou pravdepodobnosťou nefér: toto je naša **apriórna pravdepodobnosť**. Ak je prvý hod 6, naše presvedčenie, že stav1 je nefér, sa posilní (ak je hádzanie šestiek pravdepodobnejšie pri nefér kocke). Ak sa opäť hodí 6, informácia prúdi späťne od druhého hodu kockou a posilňuje naše presvedčenie o stave1 o nefér kocke ešte viac. Čím viac hodov máme, tým viac informácií prúdi späťne a posilňuje alebo kontrastuje naše presvedčenia o stave, čím ilustruje spôsob, akým informácie prúdia späťne a dopredu, aby **ovplyvnili naše presvedčenie o stavoch pri posteriórnom dekódovaní**.

Pomocou niekoľkých elementárnych úprav môžeme túto pravdepodobnosť preusporiadať do nasledujúceho tvaru pomocou Bayesovho pravidla:

$$\pi_i^* = \operatorname{argmax}_k P(\pi_i = k \mid x_1; \dots; x_n) = \operatorname{argmax}_k \frac{P(\pi_i = k; x_1; \dots; x_n)}{P(x_1; \dots; x_n)}$$

5. Posterior Decoding

Pomocou niekoľkých elementárnych úprav môžeme túto pravdepodobnosť preusporiadať do nasledujúceho tvaru pomocou Bayesovho pravidla:

$$\pi_t^* = \operatorname{argmax}_k P(\pi_t = k | x_1; \dots; x_n) = \operatorname{argmax}_k \frac{P(\pi_t = k; x_1; \dots; x_n)}{P(x_1; \dots; x_n)}$$

Pretože $P(x)$ je konštanta, môžeme ju pri maximalizácii funkcie zanedbať. Preto,

$$\pi_t^* = \operatorname{argmax}_k P(\pi_t = k; x_1; \dots; x_t) \cdot P(x_{t+1}; \dots; x_n | \pi_t = k; x_1; \dots; x_t)$$

Pomocou Markovovej vlastnosti môžeme tento výraz jednoducho zapísať nasledovne:

$$\pi_t^* = \operatorname{argmax}_k P(\pi_t = k; x_1; \dots; x_t) \cdot P(x_{t+1}; \dots; x_n | \pi_t = k) = \operatorname{argmax}_k f_k(t) \cdot b_k(t)$$

Tu sme definovali $f_k(t) = P(\pi_t = k; x_1; \dots; x_t)$ a $b_k(t) = P(x_{t+1}; \dots; x_n | \pi_t = k)$.

Ako čoskoro uvidíme, tieto parametre sa vypočítavajú pomocou dopredného algoritmu (forward algorithm) a spätného algoritmu (backward algorithm). Na vyriešenie problému posteriórneho dekódovania stačí vyriešiť každý z týchto podproblémov. Dopredný algoritmus bol ilustrovaný v predchádzajúcej časti a spätný algoritmus bude vysvetlený v nasledujúcej časti.

5. Posterior Decoding

Spätný algoritmus

Ako bolo opísané vyššie, spätný algoritmus sa používa na výpočet nasledujúcej pravdepodobnosti:

$$b_k(t) = P(x_{t+1}; \dots; x_n | \pi_t = k)$$

Môžeme začať rozvíjať rekúriu rozšírením do nasledujúceho tvaru:

$$b_k(t) = \sum_l P(x_{t+1}; \dots; x_n; \pi_{t+1} = l | \pi_t = k)$$

Z Markovovej vlastnosti potom získame:

$$b_k(t) = \sum_l P(x_{t+2}; \dots; x_n | \pi_{t+1} = l) \cdot P(\pi_{t+1} = l | \pi_t = k) \cdot P(x_{t+1} | \pi_{t+1} = k)$$

Prvý člen jednoducho zodpovedá $b_l(t+1)$. Vyjadrenie pomocou emisných a prechodových pravdepodobností dáva finálnu rekúriu:

$$b_k(t) = \sum_l b_l(t+1) \cdot a_{kl} \cdot e_l(x_{t+1})$$

Porovnanie dopredných a spätných rekúrií vedie k zaujímavým poznatkom. Zatiaľ čo dopredný algoritmus používa výsledky v čase $t-1$ na výpočet výsledku pre čas t , spätný algoritmus používa výsledky z času $t+1$, čo prirodzene vedie k ich príslušným názvom. Ďalší významný rozdiel spočíva v emisných pravdepodobnostiach; zatiaľ čo emisie pre dopredný algoritmus sa vyskytujú z aktuálneho stavu a možno ich preto vylúčiť zo sumácie, emisie pre spätný algoritmus sa vyskytujú v čase $t+1$ a preto musia byť zahrnuté do sumácie.

5. Posterior Decoding

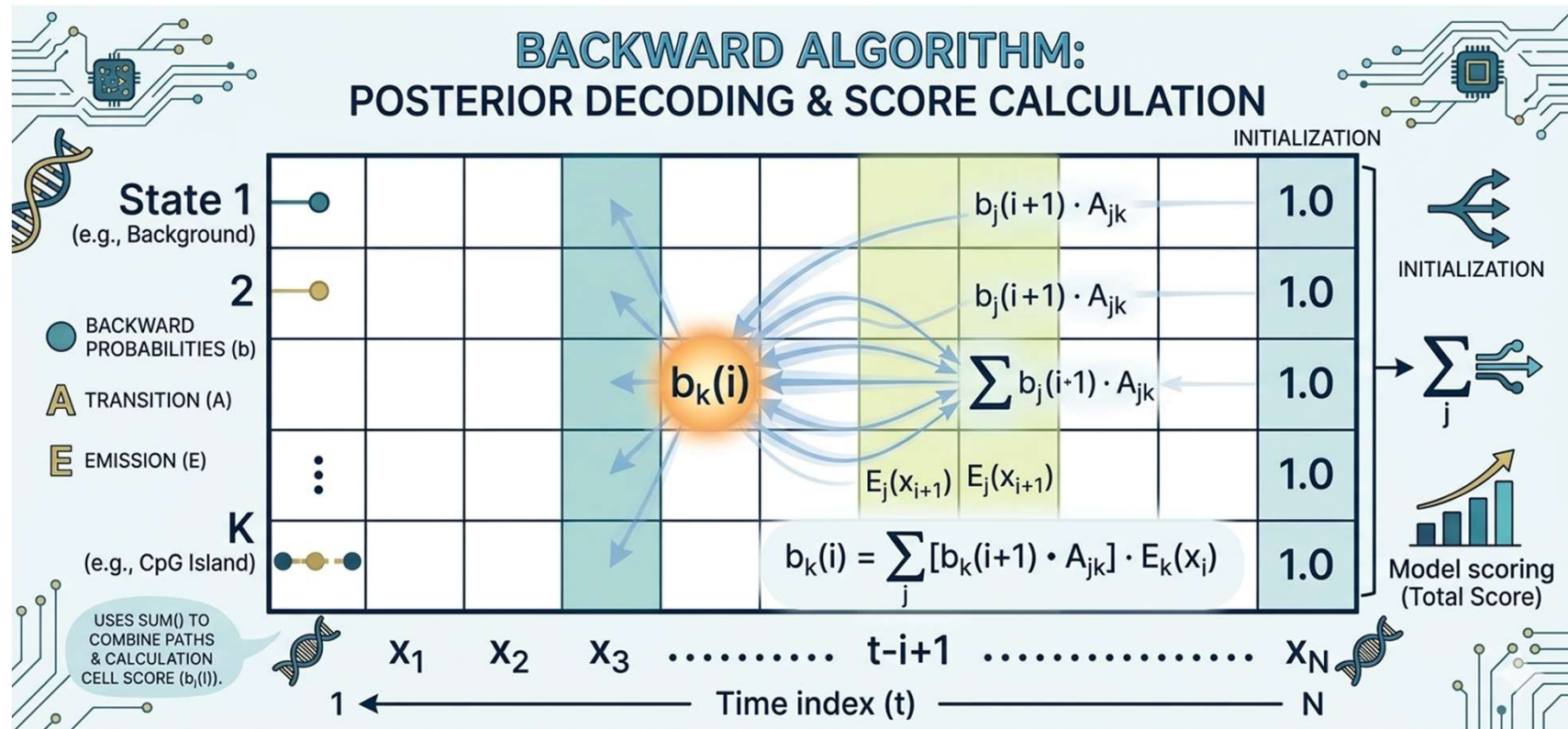
Vzhľadom na ich podobnosti nie je prekvapujúce, že spätný algoritmus je tiež implementovaný pomocou tabuľky dynamického programovania $K \times N$. Algoritmus začína inicializáciou najpravšieho stĺpca tabuľky na jednotku. Postupujúc sprava doľava, každý stĺpec sa potom vypočíta pomocou váženého súčtu hodnôt v stĺpci napravo podľa rekurzívnej uvedenej vyššie. Po vypočítaní najľavejšieho stĺpca sú všetky spätné pravdepodobnosti vypočítané a algoritmus končí. Pretože existuje KN záznamov a každý záznam skúma celkovo K ďalších záznamov, vedie to k časovej zložitosti $O(K^2N)$ a priestorovej zložitosti $O(KN)$, čo sú limity identické s limitmi dopredného algoritmu.

Rovnako ako $P(X)$ bolo vypočítané sčítaním najpravšieho stĺpca tabuľky DP dopredného algoritmu, $P(X)$ možno vypočítať aj zo súčtu najľavejšieho stĺpca tabuľky DP spätného algoritmu. Preto sú tieto metódy pre tento konkrétny výpočet prakticky zameniteľné.

Vedeli ste, že?

Všimnite si, že aj pri vykonávaní spätného algoritmu sa používajú dopredné prechodové pravdepodobnosti, t.j. ak pohyb v spätnom smere zahŕňa prechod zo stavu $B \rightarrow A$, použije sa pravdepodobnosť prechodu zo stavu $A \rightarrow B$. Je to preto, že pohyb dozadu zo stavu B do stavu A znamená, že stav B nasleduje po stave A v našom normálnom, doprednom poradí, čo si vyžaduje rovnakú pravdepodobnosť prechodu.

5. Posterior Decoding



5. Posterior Decoding

Príklad – mini kasíno

Zatiaľ čo

- **Forward algoritmus** nám hovoril: „Aká je pravdepodobnosť, že sme videli túto sekvenciu až doteraz?“
- **Backward algoritmus** sa pýta na presný opak: „Aká je pravdepodobnosť, že uvidíme zvyšok sekvencie, ak sa práve nachádzame v tomto stave?“

Opäť použijeme naše **Mini Kasíno** (Férová/F, Falošná/L) a sekvenciu $x = \{1, 6\}$.

Parametre modelu

- **Stavy:** F (Férová), L (Falošná)
- **Pozorované dáta:** $x_1 = 1, x_2 = 6$
- **Prechody (A):**
 - $P(F \rightarrow F) = 0.8, P(F \rightarrow L) = 0.2$
 - $P(L \rightarrow F) = 0.2, P(L \rightarrow L) = 0.8$
- **Emisie (E):**
 - $P(1|F) = 0.5, P(6|F) = 0.1$
 - $P(1|L) = 0.1, P(6|L) = 0.5$

Krok 1: Inicializácia (Čas $t = 2$)

Pri Backward algoritme začíname od konca. V čase $t = 2$ nemáme žiadne ďalšie informácie, preto predpokladáme, že pravdepodobnosť ukončenia je v oboch stavoch rovnaká (rovná 1).

- $b_F(2) = 1$
- $b_L(2) = 1$

5. Posterior Decoding

Krok 2: Rekúzia (Čas $t = 1$)

Teraz vypočítame hodnoty pre čas $t = 1$. Využijeme vzorec:

$$b_k(1) = \sum_{l \in \{F, L\}} a_{kl} \cdot e_l(x_2) \cdot b_l(2)$$

Keďže $x_2 = 6$, emisie $e_l(6)$ sú pre Férovú kocku 0.1 a pre Falošnú 0.5.

1. Výpočet pre $b_F(1)$ (Ak sme v čase $t = 1$ v stave Férová):

Sledujeme cesty do F a do L v čase $t = 2$:

- Cesta cez F : $a_{FF} \cdot e_F(6) \cdot b_F(2) = 0.8 \cdot 0.1 \cdot 1 = 0.08$
- Cesta cez L : $a_{FL} \cdot e_L(6) \cdot b_L(2) = 0.2 \cdot 0.5 \cdot 1 = 0.10$

Súčet:

$$b_F(1) = 0.08 + 0.10 = \mathbf{0.18}$$

2. Výpočet pre $b_L(1)$ (Ak sme v čase $t = 1$ v stave Falošná):

Sledujeme cesty do F a do L v čase $t = 2$:

- Cesta cez F : $a_{LF} \cdot e_F(6) \cdot b_F(2) = 0.2 \cdot 0.1 \cdot 1 = 0.02$
- Cesta cez L : $a_{LL} \cdot e_L(6) \cdot b_L(2) = 0.8 \cdot 0.5 \cdot 1 = 0.40$

Súčet:

$$b_L(1) = 0.02 + 0.40 = \mathbf{0.42}$$

5. Posterior Decoding

Zhrnutie výsledkov pre $t = 1$

- $b_F(1) = 0.18$
- $b_L(1) = 0.42$

Týmto sme dokončili spätný výpočet. Ak by sme chceli vedieť najpravdepodobnejší stav v čase $t = 1$ pomocou posteriórneho dekodovania, teraz by sme k týmto hodnotám $b_k(1)$ prirátali hodnoty $f_k(1)$ (Forward) a získali by sme presný obraz o tom, kde sme sa nachádzali.

Prečo je to dôležité?

Predstav si, že by si chcel vypočítať celkovú pravdepodobnosť našej sekvencie $P(x)$.

- Forward algoritmus ju počíta „spredu“ (od prvého hodu).
- Backward algoritmus ju umožňuje dopočítať „zozadu“ a je **kľúčový pre posteriórne dekodovanie** (výpočet pravdepodobnosti stavu v konkrétnom čase pri znalosti celej sekvencie).

Opäť použijeme naše **Mini Kasíno** (Férová/F, Falošná/L) a sekvenciu .

Pochopenie algoritmu **Backward** (spätného algoritmu) býva často náročnejšie než pri *Forward* algoritme, pretože intuitívne „cúvame v čase“. Poďme sa na to pozrieť úplne inak – nie cez vzorce, ale cez **tok informácie**.

Intuitívna analógia: „Čo sa stane ďalej?“

Predstav si, že kráčaš po tme a v ruke máš baterku.

- **Forward algoritmus** je ako tvoja pamäť: „Kade som išiel? Kde som bol doteraz?“ (Zhromažďuješ informáciu od štartu po prítomnosť).
- **Backward algoritmus** je ako tvoj zrak: „Kade môžem ísť ďalej? Aká je šanca, že ak pôjdem týmto smerom, uvidím cieľ?“

Algoritmus Backward sa nepýta, čo sa stalo. Pýta sa: „**Ak som teraz v stave k , aká je šanca, že zvyšok sekvencie, ktorú už poznáme, sa reálne udeje?**“

Prečo to počítame „odzadu“?

V príklade s kasínom (sekvencia 1, 6) máme dáta dopredu dané. Vieme, že po '1' nasledovala '6'.

Keď sme v čase $t = 1$ v stave F (Férová), algoritmus Backward sa „pozerá“ na budúcnosť:

1. „Ak som v F , môžem prejsť do F alebo do L .“
2. „Ak prejdem do F , aká je šanca, že z F vypadne '6' (to je emisia)?“
3. „Ak prejdem do L , aká je šanca, že z L vypadne '6'?“

Pretože nevieme, do ktorého stavu v čase $t = 2$ sme sa dostali, **musíme spočítať všetky možnosti** (preto tá suma $\sum$ vo vzorci).

5. Posterior Decoding

Rozbor vzorca pre lepší prehľad

Vzorec, ktorý ťa možno mátie:

$$b_k(t) = \sum_{l \in \{F, L\}} \underbrace{a_{kl}}_{\text{Prechod}} \cdot \underbrace{e_l(x_{t+1})}_{\text{Emisia}} \cdot \underbrace{b_l(t+1)}_{\text{Budúcnosť}}$$

Pozrime sa, čo každá časť znamená v praxi:

- a_{kl} (**Prechod**): „Šanca, že sa z aktuálneho stavu presuniem do stavu l .“
- $e_l(x_{t+1})$ (**Emisia**): „Šanca, že v tom novom stave l naozaj vznikne ten znak, ktorý vidíme v dátach (tá '6').“
- $b_l(t+1)$ (**Budúcnosť**): „Šanca, že všetko, čo má prísť *po* tomto kroku, sa podarí úspešne dokončiť.“

Prečo to vynásobíme?

Lebo všetky tieto veci sa musia stať **naraz** (a to, a zároveň to, a zároveň to).

Prečo to sčítame (suma)?

Lebo existuje viac ciest do „budúcnosti“. Môžem skončiť v stave F alebo v stave L . Obe cesty sú platné, takže ich pravdepodobnosti sčítame.

5. Posterior Decoding

Kedy tento algoritmus „ožíva“?

Samotný Backward algoritmus ti nedá odpoveď na otázku „V akom stave som bol?“. Dáva ti len kus skladačky. Tá kúzo nastáva, keď ho spojíš s Forward algoritmom:

$$\text{Pravdepodobnosť stavu } k \text{ v čase } t = \frac{f_k(t) \cdot b_k(t)}{P(\text{celá sekvencia})}$$

- $f_k(t)$ (Forward) ti povie: „Aké je pravdepodobné, že som sa sem dostal?“
- $b_k(t)$ (Backward) ti povie: „Aké je pravdepodobné, že odtiaľto úspešne dokončím túto sekvenciu?“

Keď tieto dva pohľady vynásobíš, získaš najpresnejší možný odhad stavu, aký existuje – berieš do úvahy minulosť aj budúcnosť súčasne.

Forward ti hovorí o minulosti, Backward o budúcnosti a spolu tvoria kompletný obraz o tom, čo sa v systéme deje.